

Lab #1: Basic Photon Statistics

Lab report is due Monday, Oct 5th, 2026, before 11:59 pm EDT

1. Overview

This handout provides a description of the first lab of the course. Because we use the 1st lab to introduce many new skills, from CCD fundamentals to Python programming for data analysis and visualization, as well as document writing using LaTeX, this document can hopefully serve as a straightforward guide that you can follow ~sequentially.

1.1 Schedule

This is a three-week lab, with lectures and lab sessions the weeks of September 14th, 21st, and 28th. You should aim to complete everything up to section 4 during the 1st week – go over the background, start playing with the detector and making some measurements. By the end of the 2nd week you should try to have all the data you need recorded. The 3rd lab week can be used for analysis, write-up, and re-taking any data you may need. **Do not underestimate the time the analysis and write-up will take!** The lab report is due on October 5th on or before 11:59pm EDT.

1.2 Goals

The primary goal of the 1st lab is to explore the basics of Optical / Infra-Red (OIR) photon detection, and to learn how to conduct simple statistical analyses and data visualization using Python programming language. (Check the calendar of the TA tutorials/office hours for tutorials on how to start Python programming.)

1.3 References

- **Python:** In the class web page on Quercus, you can find “*Introductory Document for Computing Resources*” by Programmer Supporter TA. The following web pages can be very useful for learning Python:
 - https://docs.scipy.org/doc/scipy/getting_started.html
 - <https://numpy.org/> (on numerical computing in Python)
 - <https://matplotlib.org/> (on visualization in Python)
- **LaTeX document writing:** You are required to write your lab reports using LaTeX in this course.
 - The Overleaf web page at <https://www.overleaf.com/> provides the service that you can use free.
- **Linux:** For those who use Python on Linux (including JupyterHub), it is important to understand basic Linux commands. There are thousands of help pages and all LLMs are well-versed, but a basic introduction is at:
 - <https://ubuntu.com/desktop/docs/en/latest/tutorial/the-linux-command-line-for-beginners/>
- Textbooks, both available in digital access at the UofT Library:
 - [Electronic Imaging in Astronomy](#), Ian S. McLean, 2008

- [Handbook of CCD Astronomy](#), S. B. Howell, 2012, 2nd edition

1.4 Key Steps

The lab is broken down into several stages:

1. Read through the entire lab manual to familiarize yourself with the plan. Please do this outside class to make efficient use of time with the equipment.
2. Work through the basic statistical, analysis, and plotting exercises here.
3. Familiarize yourself with the CCD or CMOS, check how to set gains, how to record and read back data.
4. Take data on dark and light frames.
5. Write your lab report.

2. Photon Statistics

Photons are emitted by most astrophysical sources in a random and uncorrelated way. The rate (or likelihood per unit time) of photon emission can be calculated from underlying physical models, but individual events follow Quantum Mechanical randomness. When looking at Optical and Infra-Red wavelengths, photons from natural processes are typically well-separated in time, and their generation is well described as independent and uncorrelated processes. As we saw in class, such uncorrelated events are well described by **Poisson Statistics**.

2.1 Poisson Statistics

Often astronomical signals (e.g., photons detected with an astronomical detector based on photoelectric effect) follow a Poisson probability distribution,

$$P(x; \mu) = \frac{\mu^x}{x!} e^{-\mu}$$

where $P(x; \mu)$ is the probability x events will be measured within an interval given a mean of μ events. Note that a Poisson distribution is a discrete (as opposed to continuous) distribution and cannot take a negative mean value. As shown in Figure 1 below, the Poisson distribution is an asymmetric distribution when the mean value is small, but it becomes more symmetric as the mean increases.

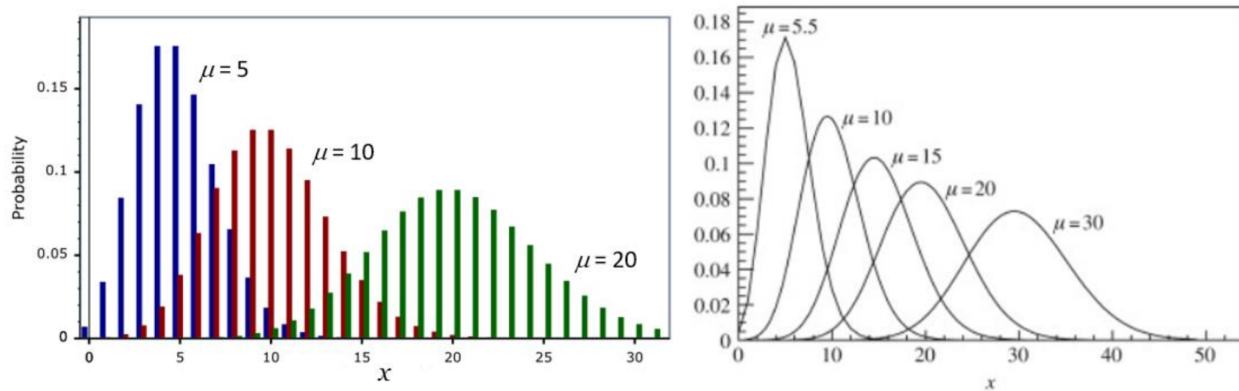


Figure 1: Poisson probability distribution for different mean values μ .

2.2 Gaussian Statistics

For large μ , the Poisson distribution approaches a Gaussian probability distribution (often called normal distribution), which takes the following form:

$$P(x; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Note that the normal distribution is continuous, and its mean can be a negative value. As in Figure 2 below, the Gaussian distribution is a symmetric distribution where $\approx 68.3\%$ of the entire probability lies within $\mu \pm \sigma$.

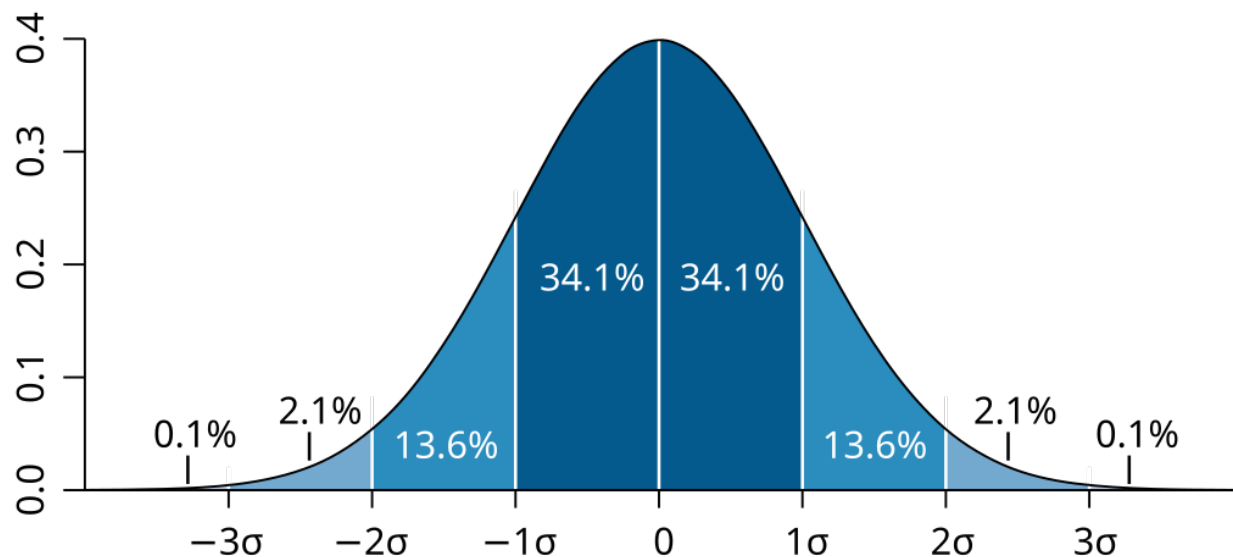


Figure 2: Gaussian probability distribution (Wikipedia)

The Gaussian probability distribution is special – and called “normal” – because of the Central Limit Theorem, which states that the sample mean (or the sum of samples) will approach a Gaussian distribution as the number of samples grows large, *regardless of the underlying distribution of samples*.

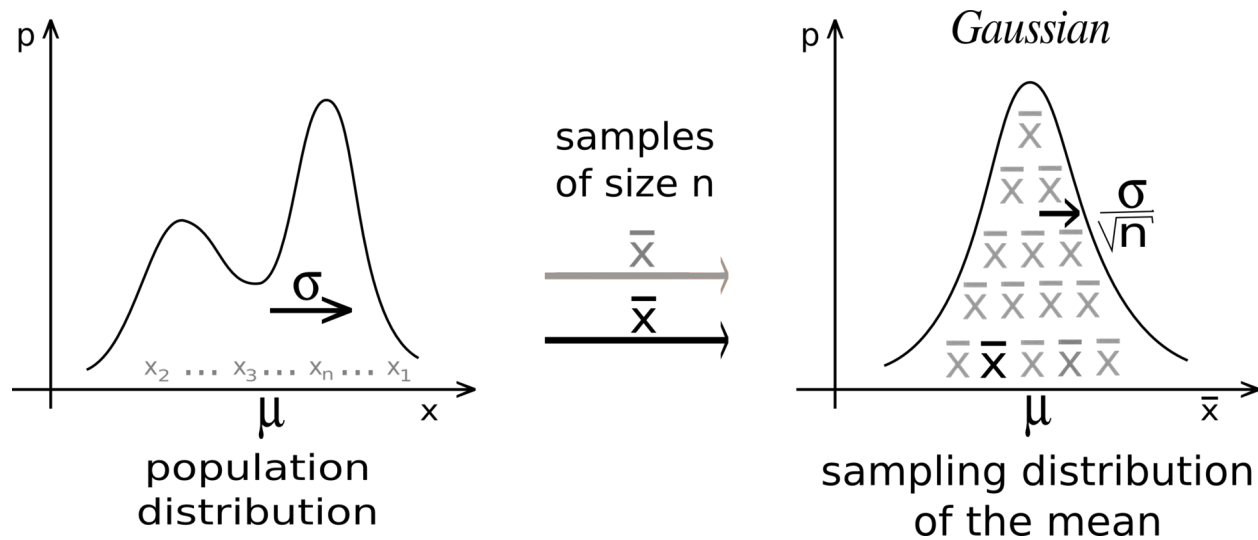


Figure 3: The central limit theorem, as visualized on Wikipedia, showing the sample mean becoming a normal distribution, regardless of the underlying distribution of individual samples.

In contrast to the Poisson distribution, the variance and standard deviation cannot be determined from the mean value in a Gaussian distribution, but depends on the variance of the underlying populations which were sampled to produce the Gaussian distribution.

$$\sigma_{\bar{x}}^2 = \sigma_{parent}^2 / n$$

2.3 Least-squares Fitting

One of the primary skills we need to learn is **model regression**, more directly called **curve fitting**. This is a broad and deep topic, and could easily fill a course (or entire statistics degree program!) on its own; we’re just going to be skimming the surface.

To make sense of an observation, we often use a **parameterized model** to describe our data. We first select a model and then adjust its parameters to fit our data. Optimizing the fit between our model and data can be complex, but a common method is choosing the **Maximum Likelihood**. An in-depth explanation is beyond the scope of this course but the main idea is to choose parameter values for our model that make it the most likely (highest probability) version of our model to produce the data we gathered.

In practice, this often reduces to a **least-squares fit**. We first define the **residuals** - the difference between the observed values and the values the model suggests. The term “least-squares” refers to choosing parameters that minimize the **square of the residuals**.

In this lab, we will first focus on linear-least squares where the model is a straight line, but we will generalize the least-squares method to other non-linear functions. It is important that in this lab *you **do not** use a canned least-squares routine* and you write your own least-square routine. In future labs, canned routines will be permitted, but it’s important to develop an understanding prior to that.

2.3.1 Fitting a Straight Line

Suppose that we have a set of N observations (x_i, y_i) where we believe that the measured value, y , depends linearly on x , so that our model is:

$$y = mx + c$$

For example, if we are measuring the motion of an object, defining y as the position and x as the time, we may start by modeling this as motion at a constant velocity, using the parameters speed (m) and initial position (c) of the object.

Given our data, what are the best estimates of m and c ? To make this concrete, let’s assume that the independent variable, time = x_i , is known exactly (or with negligible uncertainty), and the dependent variable, position = y_i , is measured with uncertainties drawn from a Gaussian probability distribution with standard deviation σ .

Under these circumstances the most likely values of m and c are those corresponding to the model which minimizes the total square deviation,

$$\chi^2 = \sum_i [y_i - (mx_i + c)]^2$$

The best values of m and c are found by solving the simultaneous equations,

$$\frac{\partial}{\partial m} \chi^2 = 0, \frac{\partial}{\partial c} \chi^2 = 0$$

Evaluating the derivatives yields

$$\begin{aligned} \frac{\partial}{\partial m} \chi^2 &= \frac{\partial}{\partial m} \sum_i [y_i - (mx_i + c)]^2 = 2m \sum_i x_i^2 + 2c \sum_i x_i - 2 \sum_i x_i y_i = 0 \\ \frac{\partial}{\partial c} \chi^2 &= \frac{\partial}{\partial c} \sum_i [y_i - (mx_i + c)]^2 = 2m \sum_i x_i + 2cN - 2 \sum_i y_i = 0 \end{aligned}$$

This set of equations can conveniently be expressed compactly in matrix form,

$$\begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & N \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix}$$

and then solved by multiplying both sides by the inverse,

$$\begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & N \end{bmatrix}^{-1} \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix}$$

The inverse can be computed analytically, or in Python it is trivial to compute the inverse numerically.

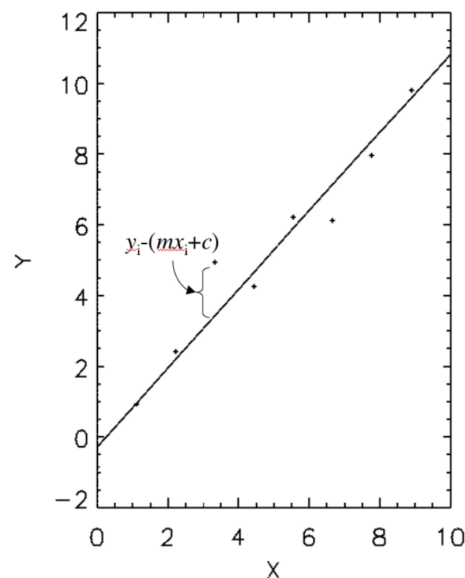


Figure 4: Example data with a least-squares fit to a straight line. A typical deviation from the straight line is illustrated.

3. Photon Detection and CCDs

At X-ray, ultraviolet, optical and infrared wavelengths, most astronomical instruments employ the photoelectric effect to convert photons into electrons, which can then be counted and recorded. Since the 1930's astronomers have used photomultiplier tubes, which employ the photoelectric effect in vacuum. Modern detectors are based on solid-state devices constructed from semiconductor materials.

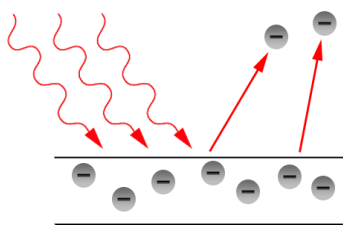


Figure 5: The photoelectric effect, in which electrons are ejected from a material following the absorption of a photon of sufficient energy.

The process of photon detection and counting is not perfect — detectors have flaws and reality has features, both of which introduce noise and various systematic errors into the measurement process. In this lab, you will use a commercial optical CCD (or CMOS) camera and learn how to characterize its properties, in particular the noise properties.

3.1 CCDs

Astronomy has for many years made use of image sensors based on CCD (Charge-Coupled Device) integrated circuits. A CCD sensor is a two-dimensional array of pixels where charges are shifted down each column one row at a time to a horizontal shift register. Charge is generated in each pixel by the photoelectric effect in proportion to the number of visible-light photons incident at that location. The charge within pixels in each row is then shifted onto a capacitor and read one pixel at a time. This voltage is then amplified by a preamplifier within the sensor and by another adjustable gain amplifier within the camera. Finally, this analog signal is digitized by an ADC (analog-to-digital converter), and the data is conveyed to a computer.

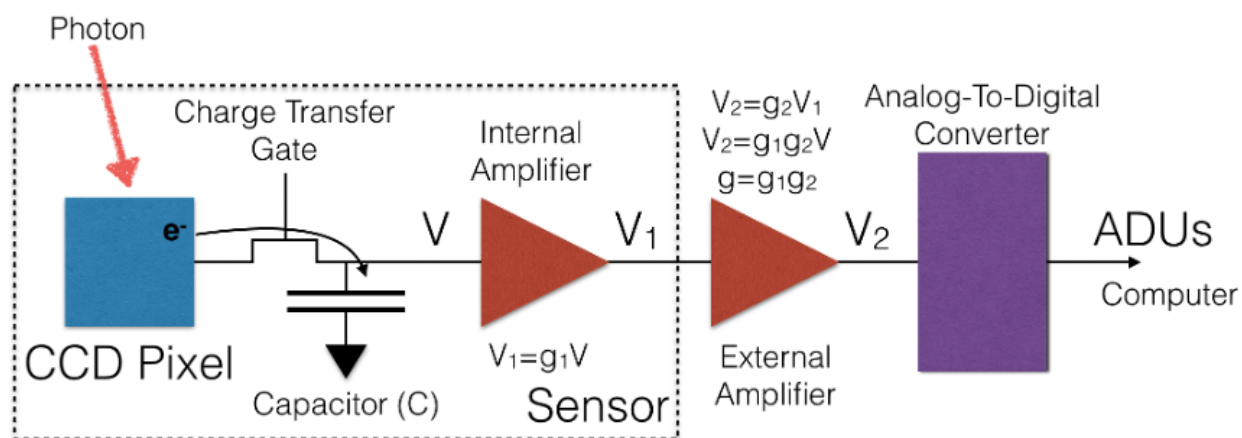


Figure 6: Analog signal chain from a CCD sensor.

The digital signal is recorded in units known as analog to digital units (ADU) after being amplified by both the sensor and the camera electronics as shown in Figure 6. Assuming that the measuring circuits are linear there is a linear relationship between ADU and the number of photoelectrons collected; this constant of proportionality is known as the **gain**. The entire CCD

array is automatically reset before the start of an exposure by clearing any remaining charge built up on the sensor. This establishes a well-defined charge state at the start of the subsequent exposure.

3.2 CMOS

An alternative and more recent imaging technology is the CMOS (complementary-symmetry metal–oxide– semiconductor) integrated circuit. A CMOS sensor is a two-dimensional array of pixels, each of which contains a photodiode, an amplifier, and some switching transistors. Each pixel is connected to external circuitry by row and column wires, so that it can be addressed, allowing the voltage on the photodiode to be measured by external circuitry elsewhere on the integrated circuit.

Charge is generated in each photodiode by the photoelectric effect in proportion to the number of photons incident at that location. Charge is accumulated for a fixed interval or exposure time. At the end of each exposure each pixel is addressed, charge-to-voltage conversion and amplification occur within each pixel, and the resultant voltage is converted to a digital signal using an ADC.

The digital signal is recorded in ADUs and each photodiode is reset at the end of an exposure by connecting it to a fixed voltage - leaving behind a well-defined charge state for the next exposure.

3.3 CMOS and CCD sources of noise

In both types of detectors, the photo-charge Q_{PE} is determined by measuring the voltage induced on a capacitor of capacitance C ,

$$V = Q_{PE} / C$$

This voltage is directly proportional to the number of incident photons.

As discussed in class, there are three main sources of noise in measurements we take using these detectors:

1. **Poisson noise:** unavoidable randomness in photon arrival times. Also called **photon noise** and **shot noise**. This is the fundamental noise associated with counting photons (or photoelectrons). The defining characteristic of a Poisson distribution is that the variance and the mean are equal. To a good approximation photons follow Poisson statistics, so the standard deviation in an experiment that counts on average N_{PE} photoelectrons is $\sqrt{N_{PE}}$.
2. **Dark current:** undesired accumulation of charge due to thermal fluctuations and quantum tunneling in the detector substrate. Electron hole pairs are created by the

photoelectric effect, but also by thermal fluctuations (phonons in the crystal lattice) and quantum tunneling. This dark current produces a systematic offset or error, which must be subtracted to find the true level of illumination. Dark current *also* contributes to Poisson noise, because the total number of charge carriers counted is increased to $N_{PE} + N_D$, where N_D is the mean number of e⁻ from dark current, and the corresponding standard deviation is $\sqrt{N_{PE} + N_D}$.

3. **Read noise:** errors generated during the process of reading out the device. This measurement error is associated with determining the voltage. For example, in an RC (Resistor-Capacitor) circuit, thermal fluctuations will induce charge variance kTC . If $C = 1$ pF and $T = 273$ K we anticipate a measurement noise of $\sigma \approx 380$ e⁻. While there are ways to circumvent read noise, there is always some statistical error associated with measuring a signal.

4. Activities

4.1 Statistics

Below is the list of steps that you need to follow for Lab #1 assignment.

- Reproduce the Figure 7 below using your own Python code and the table of measurements following it. Your plot does not have to be exactly the same in every detail but must provide a histogram compared with an expected Poisson distribution.

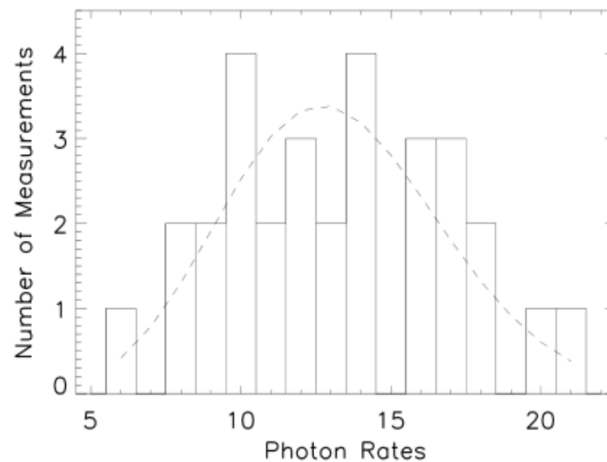


Figure 7: (Solid line) Histogram of the photon count rates; (Dashed line) Expected Poisson distribution with $\mu = 12$.

Measurements (photons / second):

13	17	18	14	11	8	21	18	9	12
9	17	14	6	10	16	16	11	10	12

8 20 14 10 14 17 13 16 12 10

- Two data files are provided on Quercus, `Large.txt` and `Small.txt`, each containing 1000 measurements of photon count rates from a star. The Small file uses a shorter integration time, so has fewer photons per datum. Independently analyze these data as follows:
 - Read the two files into Python and calculate the mean and standard deviation of the measurements recorded in each file (so you will have values for “Small” and “Large” files respectively).
 - Are your calculated values consistent with what you expect from a Poisson distribution?
 - Plot the measurements to examine their distributions. First, you can simply plot the measurements in sequence, and then use a histogram.
 - Plot a Poisson distribution on top of the histogram, as in Figure 7 above. Do the data match this distribution?
 - Overplot a Gaussian distribution as well. How well does this match? Does this fit one data set better than the other?

4.2 Camera Noise

Now let's apply what you learned to a physical experiment. *Using **basic** built-in functions in Python (e.g., `numpy.mean`, `numpy.std`, etc) is allowed if not specified otherwise.*

Half of the groups will be using a Point Grey Chameleon3 detector comprising a silicon CCD array. The other half will be using a Point Grey FMVU-03MTC-CS detector housing a CMOS sensor. Please ensure you state what detector you are using in your lab report (CCD or CMOS) and consistently use that model when conducting the data collection portion of this lab.

Table 1: Detector Properties

Specification	Chameleon3 (CCD)	FMVU-03MTC-CS (CMOS)
Sensor type	⅓-inch	⅓-inch
Model	Sony EXview HAD Monochrome CCD ICX445	Micron MT9V022 CMOS
Imaging Mode	Mono (grayscale)	Colour (RGB)
Pixels count	1288 x 964 - 1.3 Megapixels	752 x 480 - 0.3 Megapixels
Pixel size	3.75 μm	6.0 μm
Digitization	12-bit (8-bits are used)	10-bit (8-bits are used)
Gain Range	-11 to +24 dB	0 - 12 dB
Shutter Range	0.046 ms – 1/(frame rate) e.g. 33 ms @ 30 fps, ~1 s @ 1 fps	0.031 ms – 1/(frame rate) e.g. 16.6 ms @ 60 fps
Saturation Capacity	~9000 e^-	(not published)

Begin by familiarizing yourself with the camera controls and methods for acquiring images. For this activity, you need to have a 25 mm lens (Figure 8) installed on your camera to take visible-light images of your surrounding environment.



Figure 8: The C-mount 25-mm camera lens. The lens screws into the camera body. The designation C on the aperture ring means closed.

The first step of this lab is to log into a Linux laptop and start the camera viewer. Open a Terminal (the black box icon on the left of the screen) and type `lab1-viewer`, then press Enter. Plug the camera's USB cable into the laptop before you start, and take the lens cap off, or the live image

will simply be black. If more than one camera is attached, pick yours from the Camera menu at the top of the window.

The viewer window has four parts. The live image is on the left. The histogram of pixel values is at the upper right, and directly below it a statistics panel reporting the mean, standard deviation, variance, minimum and maximum of the current frame, together with the percentage of pixels that are saturated. The camera settings run along the bottom. Every number you are asked to record in the sections below is read from the statistics panel.

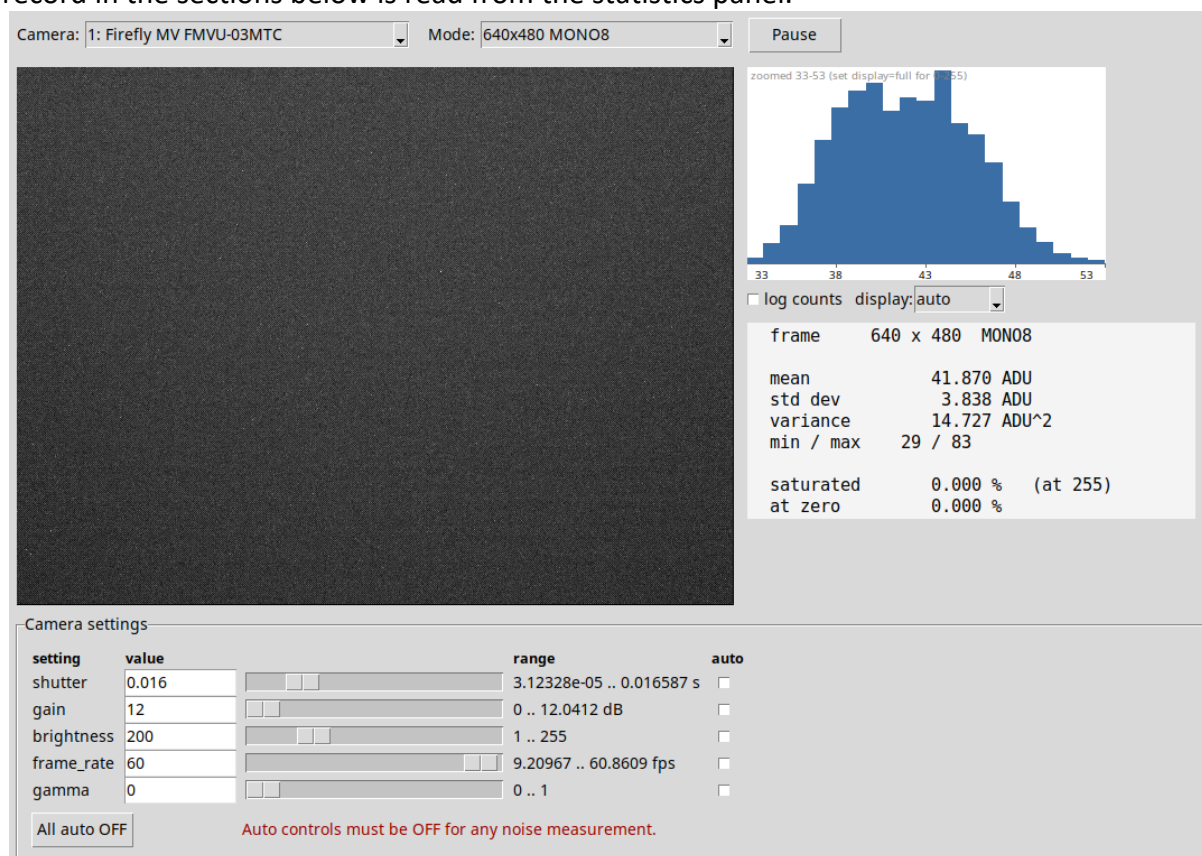


Figure 9: The Lab 1 camera viewer. Live image (left), live histogram (upper right), frame statistics (right), and the camera settings (bottom). All automatic controls are switched off when the viewer starts; leave them off for every quantitative measurement.

Focus the lens to form a sharp image of a stationary target in the room. Never touch the glass itself — adjust the lens by its focus and aperture rings only.

The settings panel along the bottom of the window lists the controls you can change: Brightness, Gamma, Shutter, Gain and Frame Rate. Each row shows the current value, a slider, and the range this particular camera allows. Drag the slider for a quick look, or type an exact number into the box and press Enter when you need a specific value — as you will in §4.2.5. Note that Shutter is given in SECONDS, so an exposure of 10 ms is entered as 0.01. The values shown are read back

from the camera, so if you ask for something it cannot do the number will snap back to what it actually used and a note will appear below the panel explaining why.

- **Brightness** adds a constant offset to every pixel value. It shifts the histogram sideways without changing its width, and it does not change how much light the sensor collects. On the CCD it is set as a percentage; on the CMOS it is a raw number from 1 to 255.
- **Exposure** is the camera's automatic-exposure target, which drives Shutter and Gain whenever their automatic controls are enabled. The viewer disables all automatic controls at startup, so this has no effect unless you deliberately switch them back on.
- **Sharpness** smooths or sharpens the image inside the camera. It must stay off for our measurements: it mixes neighbouring pixels together, which would destroy the pixel-to-pixel statistics this lab sets out to measure. The viewer leaves it off.
- **Gamma** introduces a non-linear relation between the signal measured by the ADC and the value reported for the pixel. Leave it at its default — the analysis in §4.2.4 assumes the response is linear.
- **Shutter** determines the exposure time. The exposure time can be adjusted between 0.046 ms and $1/(\text{frame rate})$ for the CCD, and 0.031 ms and $1/(\text{frame rate})$ for the CMOS — lower the frame rate first if you need a longer exposure.
- **Gain** of the ADC can be commanded and is variable from -11 to +24 dB for the CCD and 0 to 12 dB for the CMOS. This adjusts the external amplifier in the signal chain (Figure 6), and therefore the sensitivity and also the dynamic range of the camera.
- **Frame rate** sets how often the sensor is read out, and it puts a hard ceiling on the exposure time: you cannot expose for longer than one frame period. At 30 fps the longest available Shutter is about 33 ms; drop the frame rate to 1 fps and roughly a full second becomes available. If the Shutter you want is greyed out or refuses to take, lower the frame rate first and the Shutter range will widen.

For quantitative work every automatic control must be off. The viewer switches them off when it starts, and the All auto OFF button at the bottom left restores that state at any time. If an automatic control is left on, the camera will silently change the exposure or gain between frames in response to the illumination, and your noise measurements will be meaningless.

As a group, adjust each setting in turn and watch what happens to the live image, the histogram, and the statistics. Before moving on, make sure you can predict which way the histogram will move, and whether it will get wider or narrower, for each control.

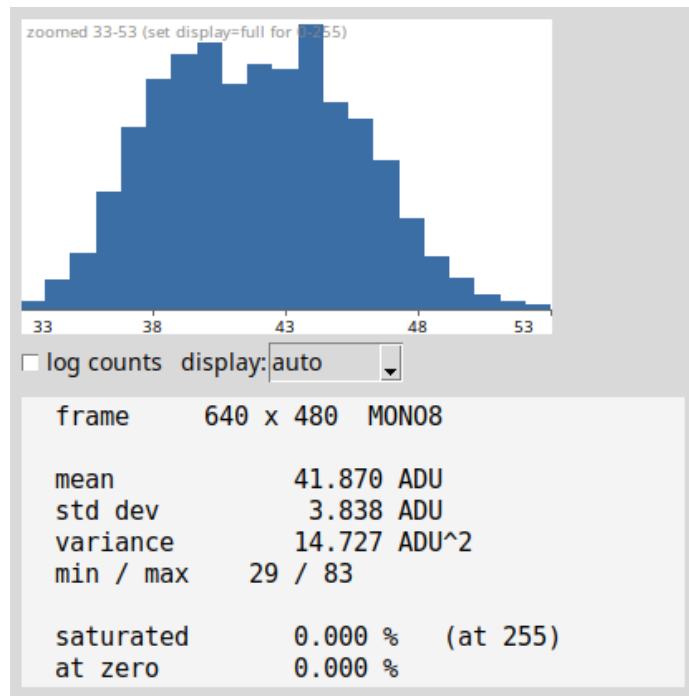


Figure 10: The histogram and statistics panel. The histogram zooms automatically to the range the data actually occupies; set display to full to see the whole 0–255 range instead. The log counts box makes rare values (such as a few hot pixels) visible.

A note for the CMOS groups: your sensor sees in colour!

The CCD is monochrome — every pixel on it measures the same thing. The CMOS is not. It carries a colour filter bonded to the sensor in a repeating 2x2 pattern, the Bayer mosaic, so each pixel measures only red, only green, or only blue. Half the pixels are green, a quarter red and a quarter blue (Figure 11a).

Nothing in the camera combines them. The frame you are given is the mosaic exactly as the sensor read it out, so a histogram of a single frame contains three populations and shows three separate peaks (Figure 11d) — one per colour. That is not a fault and it is not noise: it is three different sensitivities to the same light.

For any measurement of noise you want one population, not three. The Pixels control at the top of the viewer does this. Leave it on 'all (mosaic)' first and look at the three peaks, then switch to 'green (both)' — green because there are twice as many green pixels as either other colour.

This is why your images change shape. Selecting green keeps every green pixel and packs them together (Figure 11b and 11c): half the pixels survive, so a 752 x 480 frame becomes 752 x 240. The field of view does NOT change — the camera is still pointed at the same scene, and the

viewer scales the picture back up so it looks the same size. Only the number of samples changes. The `lab1-capture` tool saves the green pixels by default, so the files you analyse are already a single colour and already have this smaller shape. The `settings.txt` file written into each folder records the shape and the channel used; add `--channel all` to the command if you ever want the raw mosaic saved instead.

None of this changes how you analyse the data. You compute a mean and a variance for each pixel across your stack of frames, and every pixel is still one detector element measured the same way in every frame. A smaller array just means fewer points on your scatter plot — 180,480 of them, which is ample.

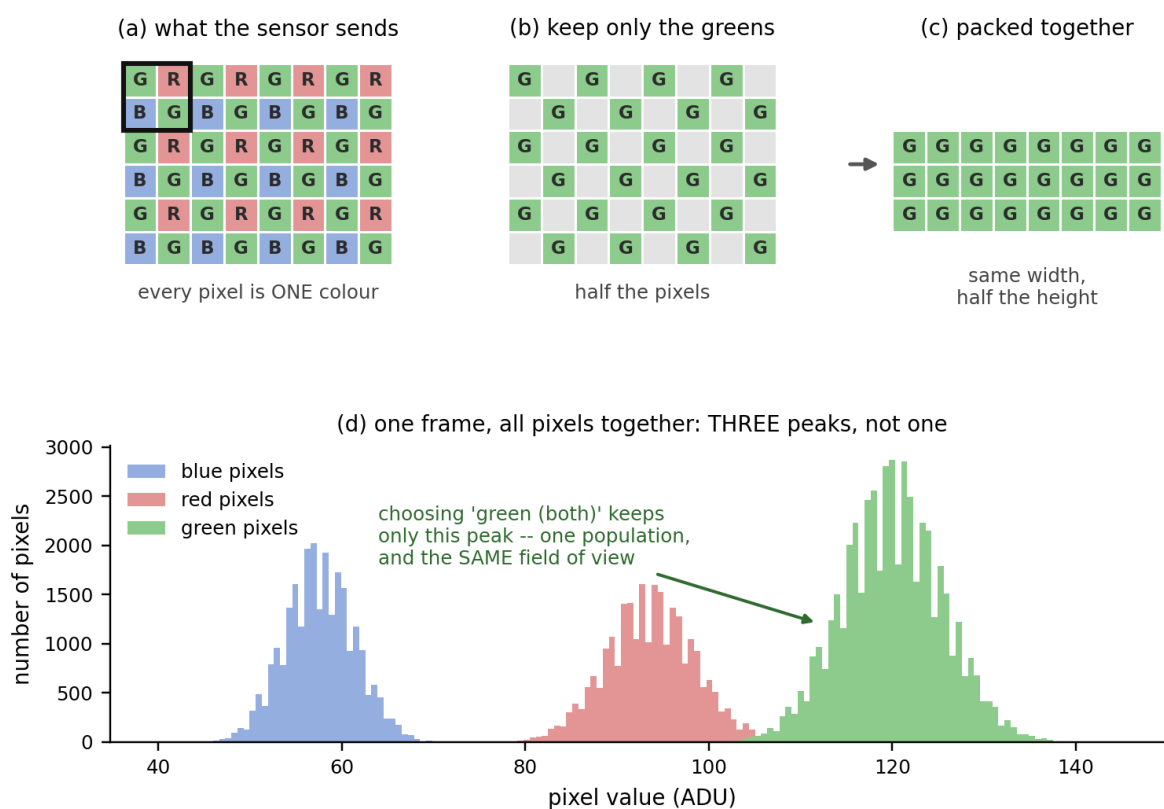


Figure 11: The CMOS sensor is a colour device. (a) A colour filter in a repeating 2x2 tile means each pixel records only one colour. (b, c) Selecting the green pixels keeps half of them and packs them together, halving the height of the array while covering the same field of view. (d) Because the three colours respond differently to the same light, a histogram of every pixel in one frame has three peaks; selecting green leaves a single population.

4.2.1 Saturation

The CCD/CMOS sensor has a limited dynamic range. With the automatic controls off, set display to full so the whole range is visible, and watch the live image, the histogram, and the statistics panel as you increase the exposure time (Shutter) from its minimum. Watch particularly the max value and the saturated percentage.

What is the maximum data value returned by the camera? What would be the maximum value for an 8-bit camera?

4.2.2 In the dark!

Place the lens cap on the camera. Set the frame rate to its lowest value by typing that number into the Frame Rate box and pressing Enter (the allowed range is printed to the right of each row). Lowering the frame rate lengthens the longest exposure the camera will allow: look at the Shutter row and you will see its range has grown.

Describe what you see in the live image as you increase the exposure time. Is it dark? Uniform? Static? By default the image is auto-scaled for display, which deliberately exaggerates faint structure — switch display to full to see the true brightness, and back to auto to hunt for detail. Look for individual bright pixels, and watch whether they get brighter as the exposure lengthens.

The statistics panel reports the mean. How does the mean value vary with exposure time?

4.2.3 Histograms

Point the camera at a uniform target. A piece of plain white paper or a clean patch of lab wall works well, especially with the lens out of focus. Check the live image for obvious structure. Set the lens aperture to F/1.4, set the gain to maximum (24 dB for the CCD, 12 dB for the CMOS), and adjust the Shutter until the mean in the statistics panel is somewhere around 180–220 ADU.

Now change only the aperture. Adjust the F-stop and watch the histogram. Describe how its shape (height, width) changes. What happens to the mean and to the standard deviation between the largest aperture (F/1.4) and the smallest (F/8)? Every camera setting must stay fixed while you do this — if you touch the Shutter or Gain the comparison is meaningless.

Now point the camera to a non-uniform source. Describe what happens to the histogram?

4.2.4 Measuring read noise

Now for some quantitative measurements! For this activity we need to save a sequence of images so that we can compute the mean and standard deviation of the signal from each pixel in the image sensor. This also allows us to explore the relationship between signal (the pixel's mean) and the noise (the pixel's standard deviation).

Point the camera at a scene with both bright and dim regions. Adjust the settings so the dimmer regions sit close to zero signal and the brighter regions come close to saturation without reaching it — the statistics panel should show a max near, but below, 255, and a

saturated percentage of 0.000. Focus is not critical, but a badly out-of-focus image blurs the bright and dark areas into uniform grey and gives you a poor range of values to work with.

As we want to explore the statistical properties of a sequence of images, choose a stationary object. A constant illumination to the detector is imperative for this experiment! If someone turns the lights on or off, you will need to start again; if someone walks in front of the camera then the changes you record will not be due to intrinsic fluctuations in a pixel.

Take some test exposures to confirm you have a good spread of pixel values, with the automatic controls off. If the darkest parts of the image are sitting at exactly 0 ADU, raise Brightness a little until they lift clear of zero: values clipped at 0 (or at the top of the range) are not measurements, and they will bias the fit in the analysis below. Brightness adds a constant and does not change the fluctuations you are trying to measure.

Now collect a sequence of images — 20 to 30 is plenty — using the capture tool. Leave the viewer running so you can see what the camera sees, and open a SECOND Terminal window to run the capture command. More frames give a better estimate, but if you take too long the illumination may drift.

In the second Terminal, type the following on one line, replacing groupA with your own group letter:

```
lab1-capture --dir light_groupA --n 30
```

The tool prints the settings it is using, then saves the frames. It writes three things into the folder: the images themselves as uncompressed TIFF files; a file `images.txt` listing every image file, which your analysis code will read; and a file `settings.txt` recording the camera settings actually in force at the time. Include the contents of `settings.txt` in your report — it is the observing log for this measurement. If you want to fix the exposure and gain explicitly rather than using whatever the viewer was set to, add for example `--shutter 0.01 --gain 0` (shutter in seconds, gain in dB).

Now repeat the experiment with the lens cap in place, saving to a different folder:

```
lab1-capture --dir dark_groupA --n 30
```

Use exactly the same exposure time and gain as your light frames; if you changed anything, take the light frames again. We need these dark images so that we can subtract them from the light frames, leaving a mean value that reflects only how much light actually fell on each pixel.

Copy both folders onto a USB stick (or your own laptop) to carry out the analysis in Python. Proceed as follows:

- 1) Each folder contains a file `images.txt` listing every image file that was saved. Write a function that reads that list, then loop over it importing your TIFF files as 2D numpy arrays, and stack them into a single 3D numpy array.

- a) An easy way to import your TIFF files is by using PIL (Python Image Library) see example code below.

```
import numpy as np
import PIL
from PIL import Image

img = Image.open('your_image.tif')

img_arr = np.array(img)      # Transform into numpy
                              array

img_arr.shape                # Check dimensions

img_arr                      # Print array
```

If you don't have PIL you can run the command `!pip install pillow`

- 2) Next, calculate the mean of each pixel for the dark frames.
- 3) Subtract the averaged dark frame from each light frame.
 - a) Write a loop!
- 4) Calculate the mean and variance of each pixel.
 - a) Numpy functions `mean()` and `var()` are great for this especially if you define an axis (For example, `mean(array, axis = 0)`).
- 5) Plot the mean and variance data using a scatter plot.
 - a) To make things easier you could reduce your 2D arrays into 1D arrays using `flatten()`.
 - b) As a suggestion, don't plot all pixels! Randomly choosing 1000 should be enough.
- 6) Carry out linear regression analysis to determine the gain and noise of your camera.
 - a) Hint: Use Least Squares Method!

After running your code answer the following questions:

- Describe the trend revealed by the scatter plot of mean pixel values and their variance, i.e., standard deviation squared.
- Why is the variance not zero when the signal is zero?
- What happens to the relation between mean and variance at high signal levels?
- What is the meaning of the slope of the graph?

What is the read noise of your camera? What is the gain?

Hint: the "gain" you're trying to calculate is the conversion factor from ADU to e^- , so it's a number in e^-/ADU .

4.2.5 Measuring Dark Current

As defined above, dark current arises from a build-up of electrons from thermal excitation and quantum tunneling in the semiconductor. For data processing, we often take dark images at the same exposure time as our light images for dark subtraction. However, since dark current also contributes to Poisson noise, dark subtraction is not perfect. Detectors are often cooled to minimize the dark current to decrease the thermal excitation and therefore lower the noise. Note that in our setup the detector is at room temperature, $\approx 300\text{K}$.

To measure the dark current you need the mean value of a series of dark frames taken with increasing exposure time. Put the lens cap on. In the viewer, switch every automatic control off, then set:

CCD: Brightness = 1.4 % (its minimum), Gain = 24 dB (max), Frame Rate = 30 fps (max).

CMOS: Brightness = 146, Gain = 12 dB (max), Frame Rate = 30 fps (max).

Now type each exposure time into the Shutter box in turn — 1, 5, 10, 15, 20, 25 and 30 ms — pressing Enter after each, and record the mean from the statistics panel. Remember that Shutter is in seconds, so 1 ms is entered as 0.001 and 30 ms as 0.03. Let the camera settle for 10–20 seconds at each step before reading the mean, and watch that the reading has actually stopped drifting before you write it down.

Plot the mean value against exposure time, do a linear least-squares fit to your data, and determine the dark current of the camera in ADU/sec/pixel.

What is the dark current of the detector in e^- per second per pixel?